

DEPARTAMENTO DE DESARROLLO

Soc Asesores



Documento:

Documentación Técnica de Proyecto Biblioteca virtual

Presenta:

Ingeniero Uriel Cisneros Torres

Fecha:

15 de Mayo del 2026, Casimiro Castillo Jalisco

1. Tecnologías Utilizadas.....	3
2. Estructura del Proyecto.....	4
3. Endpoints de la API.....	5
3.1 Autenticación.....	5
3.2 Usuarios.....	5
3.3 Carpetas.....	5
3.4 Archivos.....	5
3.5 Logs / Bitácora.....	6
4. Cómo Ejecutar el Proyecto.....	7
5. Configuración (appsettings.json).....	8

1. Tecnologías Utilizadas

Stack completamente .NET moderno con separación total de responsabilidades.

Tecnología	Versión	Rol en el sistema
.NET 10 / ASP.NET Core	net10.0	Framework principal para la API REST. Middleware pipeline, contenedor DI, autenticación JWT integrada, Hosted Services.
Entity Framework Core 10	10.0.0	ORM para mapeo objeto-relacional. Migraciones, Fluent API de configuración, seeding automático de datos iniciales.
SQLite	EF Core Sqlite 10.0.0	Base de datos relacional embebida. Archivo único biblioteca.db, sin servidor adicional. Ideal para entornos de desarrollo.
JWT Bearer Auth	System.IdentityModel.Tokens.Jwt 8.3	Tokens firmados con clave simétrica HS256. Claims incluyen correo, perfil y permiso de gestión de archivos.
BCrypt.Net-Next	4.0.3	Hashing seguro de contraseñas con salt automático. Work factor configurable para balance seguridad/rendimiento.
FluentValidation	vía Application layer	Validación declarativa de DTOs: longitudes, formatos de correo, campos requeridos. Reglas encadenadas y reutilizables.
Swagger / OpenAPI	Swashbuckle	Documentación interactiva en /swagger. Soporte para autenticación Bearer, descripción de endpoints y esquemas.
Background Service	IHostedService / .NET	LimpiezaArchivosService: ejecuta cada 60 segundos y elimina físicamente archivos borrados lógicamente tras el período configurado.
Clean Architecture	Patrón de diseño	Separación en 4 proyectos: Domain (núcleo puro), Application (casos de uso), Infrastructure (implementaciones), API (presentación).

2. Estructura del Proyecto

Monorepo con solución .NET organizada en tres proyectos src/ y una carpeta tests/.

BibliotecaVirtual.sln

src/

└─ BibliotecaVirtual.Domain/

- | └─ Common/BaseEntity.cs
- | └─ Entities/ - Usuario, Carpeta, Archivo, LogAccion, HistorialUsuario
- | └─ Enums/ - TipoPerfil, TipoArchivo, TipoAccionLog
- | └─ Exceptions/DomainException.cs
- | └─ Interfaces/ - IUserarioRepository, ICarpetaRepository, IArchivoRepository, ILogAccionRepository, IUnitOfWork

└─ BibliotecaVirtual.Application/

- | └─ Usuarios/ - Contracts, Dtos, Services (AuthService, UsuarioService), Validators
- | └─ Carpetas/ - Contracts, Dtos, Services (CarpetaService), Validators
- | └─ Archivos/ - Contracts, Configuration, Dtos, Services (ArchivoService), Validators
- | └─ Logs/ - Contracts, Dtos, Services (LogService)
- | └─ Common/ - Result<T>, CurrentUserContext
- | └─ DependencyInjection.cs

└─ BibliotecaVirtual.Infrastructure/

- | └─ Persistence/ - AppDbContext, UnitOfWork, DbSeeder, CarpetaSeeder, Configurations/
- | └─ Repositories/ - UsuarioRepository, CarpetaRepository, ArchivoRepository, LogAccionRepository
- | └─ Identity/ - JwtTokenGenerator, JwtSettings, BCryptPasswordHasher
- | └─ Storage/AlmacenamientoArchivos.cs - I/O físico de archivos en disco
- | └─ BackgroundServices/LimpiezaArchivosService.cs - tarea periódica de limpieza
- | └─ DependencyInjection.cs

└─ BibliotecaVirtual.API/

- | └─ Controllers/ - AuthController, UsuariosController, CarpetasController, ArchivosController, LogsController, HealthController
- | └─ Middleware/ - ExceptionHandlingMiddleware, CurrentUserMiddleware
- | └─ Program.cs - Entry point, pipeline, migraciones y seed automático
- | └─ appsettings.json

```
tests/  
└─ BibliotecaVirtual.Tests/  
    │ └─ UsuarioServiceTests.cs  
    │ └─ Helpers/TestContextFactory.cs
```

3. Endpoints de la API

Todos los endpoints requieren Authorization: Bearer {token}, excepto POST /api/auth/login.

3.1 Autenticación

Método	Ruta	Descripción	Roles
POST	/api/auth/login	Autenticación con correo y contraseña. Retorna JWT.	Público

3.2 Usuarios

Método	Ruta	Descripción	Roles
POST	/api/usuarios	Crear nuevo usuario.	Admin, Gerente
GET	/api/usuarios	Listar todos los usuarios.	Admin, Gerente
GET	/api/usuarios/{correo}	Obtener usuario por correo.	Admin, Gerente
PATCH	/api/usuarios/{correo}/inactivar	Inactivar usuario.	Admin, Gerente
PATCH	/api/usuarios/{correo}/reactivar	Reactivar usuario inactivo.	Admin, Gerente
PATCH	/api/usuarios/{correo}/permisos	Otorgar permisos a Asistente.	Admin, Gerente
GET	/api/usuarios/{correo}/historial	Ver historial de acciones del usuario.	Admin, Gerente

3.3 Carpetas

Método	Ruta	Descripción	Roles
GET	/api/carpetas	Listar carpetas raíz.	Todos los usuarios

GET	/api/carpetas/{id}	Obtener carpeta con subcarpetas y archivos.	Todos los usuarios
POST	/api/carpetas	Crear carpeta o subcarpeta.	Admin, Gerente, Asistente (con permiso)

3.4 Archivos

Método	Ruta	Descripción	Roles
GET	/api/archivos/carpeta/{carpetaid}	Listar archivos activos de una carpeta.	Todos los usuarios
GET	/api/archivos/{id}	Obtener metadata de un archivo.	Todos los usuarios
POST	/api/archivos/subir	Subir archivo (multipart/form-data, máx. 300 MB).	Admin, Gerente, Asistente (con permiso)
PATCH	/api/archivos/{id}	Editar nombre y descripción del archivo.	Admin, Gerente, Asistente (con permiso)
DELETE	/api/archivos/{id}	Eliminación lógica del archivo.	Admin, Gerente, Asistente (con permiso)
GET	/api/archivos/{id}/descargar	Descargar el archivo físico.	Todos los usuarios

3.5 Logs / Bitácora

Método	Ruta	Descripción	Roles
GET	/api/logs	Listar todas las acciones registradas.	Admin, Gerente

4. Cómo Ejecutar el Proyecto

El proyecto no requiere ningún servidor de base de datos externo. Solo es necesario el SDK de .NET 10.

#	Paso	Detalle
1	Verificar prerequisitos	Instalar el SDK de .NET 10 desde https://dotnet.microsoft.com/downloadaddotnet --version # Debe mostrar 10.x.x
2	Clonar / Abrir el proyecto	cd ExamenNet2/ExamenNet2Is # Debe mostrar BibliotecaVirtual.sln
3	Restaurar paquetes NuGet	dotnet restore
4	Compilar la solución	dotnet build
5	Ejecutar la API	cd src/BibliotecaVirtual.APIdotnet run# Salida esperada:# Now listening on: http://localhost:5000Al iniciar, el sistema aplica migraciones y ejecuta el seeding automáticamente(usuario admin + carpetas raíz predeterminadas).
6	Acceder a Swagger UI	http://localhost:5000/swaggerInterfaz interactiva para probar endpoints.Usar el botón "Authorize" con: Bearer {tu-token}
7	Ejecutar pruebas unitarias	dotnet test

Usuario inicial (seed): `admin@biblioteca.com` / contraseña: **Admin123!** — [Cambiar en producción.](#)

5. Configuración (appsettings.json)

Archivo: src/BibliotecaVirtual.API/appsettings.json

Clave	Valor por defecto	Descripción
ConnectionStrings:DefaultConnection	"Data Source=biblioteca.db"	Ruta del archivo SQLite
Jwt:Key	"<clave-mínimo-32-caracters>"	Clave secreta para firmar JWT
Jwt:Issuer	"BibliotecaVirtual"	Emisor del token
Jwt:Audience	"BibliotecaVirtualClientes"	Audiencia del token
Jwt:ExpiresInMinutes	60	Tiempo de expiración en minutos
Almacenamiento:RutaBase	"uploads"	Carpeta de almacenamiento físico
Almacenamiento:MaxTamanoDocumentosBytes	52428800	Máx. documentos: 50 MB
Almacenamiento:MaxTamanoMediaBytes	314572800	Máx. video/audio: 300 MB
CarpetasIniciales:Bancos	["Banco A","Banco B",...]	Bancos para carpetas raíz iniciales